Community Security Alert System (CSAS)

Systems Design Report

Workshop No. 2 — Revised Submission

Authors	Felipe José Garzón Herrera	*20251020132*
	Juan Esteban Quintero Gordillo	*20251020137*
	Henry Samuel Garrido Medina	*20251020125*
	Gabriel Mateo Cusba Marín	*20251020128*
Course	Systems Analysis and Design	
Professor	Eng. Carlos Andrés Sierra, M.Sc.	
Program	Computer Engineering	
Institution	Universidad Distrital Francisco José de Caldas	
Semester	2026-I	

Document type: Systems Engineering Technical Report. This revision addresses professor feedback on the first submission and converts the document from academic-paper format to systems engineering report format per workshop guidelines.

Contents

1. Introduction and Design Rationale

This document constitutes the systems design report for the Community Security Alert System (CSAS), a socio-technical intervention on the campus safety system of Universidad Distrital Francisco José de Caldas. The design is grounded exclusively in the empirical findings of Workshop No. 1, which characterised campus safety as a system bounded not by the willingness of its members to act but by the *latency*, *reliability*, and *traceability* of the information flow connecting incident, awareness, and response.

Table 1 restates the operational gaps from Workshop No. 1 that drive every design decision in this document.

Table 1: Operational Gap Analysis — W1 Baseline and W2 Design Targets

Dimension	Baseline (W1)	Design target	Gap
Incident-to-response time	≈11.4 min	<5 min	−6.4 min
Incident record completeness	≈41% of events	≥85%	+44 pp
Community channel adoption	≈12%	≥60%	+48 pp
Alert delivery success	≈87% (phone/SMS)	≥99.9%	+12.9 pp

The document is organised as a systems engineering report, not an academic paper: it prioritises traceable design decisions, operational workflows, system architecture (people + processes + technology), and concrete usage scenarios over abstract description.

2. Requirements Analysis

2.1. Functional requirements. — Table 2 lists the ten functional requirements. Each is explicitly traced to the W1 empirical finding that motivates it, operationalising the principle that requirements must emerge from system analysis.

2.2. Non-functional requirements. —

- **NFR-01 (Latency):** End-to-end latency from submission to alert broadcast <30 s (95th percentile).
- **NFR-02 (Reliability):** Notification delivery ≥99.9% across all channels.
- **NFR-03 (Availability):** System uptime 99.9%; guaranteed availability 18:00–22:00 (highest-risk window per W1 observation).
- **NFR-04 (Privacy):** Full compliance with Ley 1581 de 2012; TLS 1.3 for all data in transit.
- **NFR-05 (Scalability):** API gateway sustains a 300% concurrent-request surge without performance degradation.
- **NFR-06 (Usability):** Reporting workflow completable in ≤3 interactions; onboarding in ≤2 minutes.

Table 2: Functional Requirements with W1 Traceability

ID	Requirement	W1 evidence that motivates it
FR-01	Users shall report incidents by type, GPS location, and description in $\leq$3 interactions.	High-friction reporting identified as root cause of under-reporting (interview + observation).
FR-02	Real-time geofenced alerts shall broadcast to users within 500 m of a verified incident.	Spatial concentration of incidents in specific zones (observation: 2/2 suspicious events in parking lot at night).
FR-03	An AI plausibility scorer shall filter implausible reports before human review.	Need to mitigate alert fatigue (Balancing Loop B1, W1 CLD).
FR-04	Security staff shall have a web dashboard to review, escalate, and resolve reports.	Staff interviews: no centralised incident management tool exists.
FR-05	Community members shall confirm or flag reports submitted by others.	100% of incident-experienced survey respondents willing to participate (W1 survey, $n = 20$).
FR-06	The mobile client shall capture GPS coordinates automatically on report submission.	Observer note: verbal descriptions insufficient for zone-level routing; exact location needed.
FR-07	Every incident shall be persisted in an auditable log with timestamps and outcomes.	Institutional document analysis: formal records capture only $\approx$41% of actual events.
FR-08	A one-touch SOS button shall trigger immediate dispatch bypassing verification.	Interview: average $\approx$11.4-min latency incompatible with life-safety emergencies.
FR-09	The analytics engine shall generate zone-level security heatmaps for university management.	Benchmarking: Policía Nacional / SIURE provide no real-time analytical layer.
FR-10	Users shall attach image or video evidence to reports.	Observation: suspicious events went unrecorded by other witnesses; media adds verifiability.

- **NFR-07 (Maintainability):** Any service component shall be independently updatable.
- **NFR-08 (Integrity):** No alert dispatches as verified without $\geq$2 community confirmations *or* 1 administrative approval.

3. Concept of Operations (CONOPS)

Before specifying architecture, it is essential to describe *how the system operates from a human perspective*: who does what, under what conditions, and with what information.

This section fulfils the systems engineering obligation to model the interaction of people, processes, and technology before defining the technology itself.

3.1. Operational roles. — Table 3 defines the four roles that execute the operational workflows described below.

Table 3: Operational Roles in the CSAS Environment

Role	Description	Primary system interaction
Community member	Any student, professor, or staff member with a registered account	Submits reports, receives geofenced alerts, confirms or flags reports from others
On-duty security guard	Campus security team member scheduled for a shift	Receives high-priority and emergency alerts; approves or rejects pending reports; coordinates ground response
Security supervisor	Duty officer responsible for escalation decisions	Manages the administrative dashboard; escalates verified incidents to local authorities
System administrator	IT staff maintaining the platform	Manages accounts, monitors health, accesses audit logs under controlled protocol

3.2. Normal incident workflow. — Fig. 3.2 presents the CONOPS swim-lane diagram covering the standard incident lifecycle from report submission to resolution. The diagram makes the interaction between human processes (left lanes) and technological components (right lanes) explicit.

3.3. Emergency SOS workflow. — When a user activates the SOS button (FR-08), the normal verification pipeline is bypassed entirely. GPS coordinates are captured automatically (FR-06), an immediate dispatch message is pushed to all on-duty security guards, and a 90-second countdown begins. If no guard acknowledges the alert within 90 s, the system automatically escalates to local authorities via the secure REST endpoint. This bypass design is justified by the W1 finding that the current 11.4-minute latency is incompatible with life-safety emergencies.

3.4. Operational conditions and constraints. — Three environmental constraints shape every operational decision: (a) **Night-time window** 18:00–22:00 has the highest observed incident concentration (W1 direct observation: both suspicious events in this window); the system must remain fully staffed and fully responsive during this window (NFR-03). (b) **Connectivity dead zones**: the W1 observation phase did not assess mobile signal coverage; offline queuing is therefore mandatory. (c) **Staff availability**: a 90-second moderator timeout prevents the verification pipeline from stalling when no moderator is logged in, escalating automatically to the on-call path.

4. System Architecture

A systems architecture is not the same as a software architecture. This section describes the complete system: stakeholders, their interfaces with the technology, information flows between organisational and technological components, and the relationships between

the system and its environment. The software architecture (Section 5) specifies the technological components in isolation.

4.1. System context. — Fig. 1 defines the system boundary. Everything inside the boundary is managed, designed, and operated by the CSAS project. Everything outside the boundary is the environment: external actors, pre-existing platforms, and physical infrastructure.

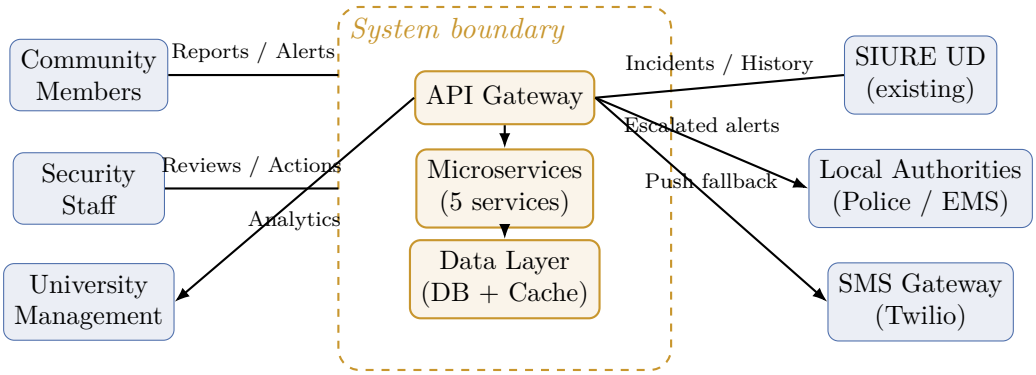

Figure 1: System context diagram. Blue boxes are external actors. Gold boxes are system-managed components. Bidirectional arrows indicate information flows in both directions. Source: own elaboration (2026).

4.2. Information flows between organisational and technological components. — Table 4 maps each information flow to the organisational sender, the system component that handles it, and the downstream organisational receiver. This table answers the professor's requirement to show how human processes interact with the technological platform.

4.3. Inputs, outputs, and constraints. — Table 5 specifies the system-level inputs, outputs, and constraints, required for any compliant systems engineering description.

5. Software Architecture

5.1. Architectural style. — The CSAS adopts a **microservices, event-driven architecture**. This choice is defended in DDR-03 (Section 6); briefly, the verification, notification, and analytics subsystems have independent load profiles and scaling needs that make a monolithic deployment suboptimal, and the alert dispatch must be decoupled from incident ingestion to prevent cascading delays during high-volume periods.

5.2. Five-layer organisation. — The architecture is organised across five layers: (i) **Presentation** — mobile app (React Native) and web portal; (ii) **Application** — API gateway (Node.js / Nginx) for authentication, rate limiting, and service routing; (iii) **Service** — five domain microservices (Table 6); (iv) **Data** — PostgreSQL + PostGIS database, Redis cache, and media object store; (v) **Integration** — bridges to SIURE UD, local authorities REST endpoint, and Twilio SMS gateway.

Table 4: Organisational–Technology Information Flows

Flow	Org. sender	System component	Org. receiver
Incident report	Community member	Incident Service → Verification Engine	Security guard (alert)
SOS activation	Community member	Incident Service (bypass)	Security guard + Authorities
Report confirmation	Community member	Verification Engine (crowd logic)	Security guard (decision support)
Alert broadcast	System (Dispatcher)	Push / SMS engine	Community members in 500 m
Incident escalation	Security supervisor	Integration layer → REST endpoint	Local authorities (Police / EMS)
Historical feed	SIURE UD (existing)	Integration layer → Analytics	Security supervisor, Mgmt
Heatmap report	System (Analytics)	Analytics engine → Dashboard	University management
Audit log	System	Audit service	System administrator

5.3. Technology stack. — Table 7 presents the recommended technology stack. Every choice cites the requirement it satisfies; the deeper engineering justification is in the DDRs.

5.4. External integrations. — Three external integrations extend the system's reach beyond the campus platform: **SIURE UD** (bidirectional REST bridge) provides the historical baseline and complements the existing institutional workflow without replacing it; **Local authorities** (outbound secure REST endpoint) activated only for high-severity escalations; **SMS gateway** (Twilio) functions as both a delivery fallback and an alternative submission channel for users without smartphone access.

6. Design Decision Records

A design decision record (DDR) documents *why* a specific architectural choice was made: what alternatives were considered, what criteria drove the evaluation, and under what conditions the chosen approach could fail. This section provides the engineering justification that was absent from the first submission.

6.1. DDR-01 — Hybrid AI + Human Verification Pipeline. — Decision: Implement a two-stage verification pipeline combining an AI plausibility scorer (Stage 1) with human moderator review (Stage 2).

Evidence driving the decision: W1 sensitivity analysis identified that "if too many alerts are sent, users may start ignoring them" (Balancing Loop B1). Any single-stage

Table 5: System Inputs, Outputs, and Operational Constraints

Category	Items
Inputs	User registration data; incident reports (type, GPS, timestamp, media); community confirmation signals; SOS activations; SIURE UD historical feed; weather API data; guard acknowledgement signals
Outputs	Geofenced push alerts; SMS fallback alerts; administrative dashboard events; heatmap reports; incident audit log entries; escalation messages to authorities
Operational constraints	Ley 1581/2012 data protection; TLS 1.3 encryption; 99.9% uptime SLA; ≤3-interaction reporting (NFR-06); maximum 45-second verification budget; 90-second escalation timeout
Environmental constraints	Unreliable mobile connectivity in parts of campus (assumption unvalidated, see Section 12); limited security staff availability outside business hours; no existing standardised incident classification schema

mechanism that is either too permissive (false positives trigger B1) or too strict (genuine reports suppressed, Loop R1 stalls) creates a failure mode.

Alternatives considered: Table 8 evaluates four options on five criteria.

Selection rationale: AI-only has no accountability and silent failure modes. Human-only cannot meet the <30 s latency requirement (NFR-01) during off-hours when staff availability is low. Crowd-only degrades during nights and early mornings precisely when the W1 data shows the highest risk (all suspicious events were during evening/night). Hybrid combines the speed of AI pre-filtering with the authority and accuracy of human decision-making for uncertain cases.

Conditions of failure: (1) AI model quality degrades due to dataset drift; mitigated by monthly retraining on verified CSAS events. (2) No moderator available during the 45-second verification budget; mitigated by an automatic timeout that escalates to the on-call security guard. (3) Community confirmations arrive slowly for low-population off-hours; mitigated by the staff override path that bypasses the crowd requirement.

How effectiveness will be measured: False-positive rate on verified alerts (target: <5%); average verification latency (target: <45 s); daily human-override rate as a proxy for AI quality.

6.2. DDR-02 — Zone-Weighted and Time-Weighted Priority Dispatch. — Decision: Compute a priority score $P = S_{\text{base}} \times W_z \times W_t$ where S_{base} is the incident severity class, W_z is the zone weight, and W_t is the time-of-day weight, before deciding which queue tier and notification radius to use.

Evidence driving the decision: W1 identified that "alert activity is not uniformly distributed across the campus or the day" and that "the system should apply zone-weighted and time-weighted priority logic rather than treating all incoming reports identically." Specifically: the parking lot was the highest-risk zone across all observation sessions; both

suspicious events occurred at night; both coincided with rainy conditions.

Model definition: Table 9 specifies the calibrated weight values. Initial values are derived from W1 observations; they will be revised quarterly using incident analytics (FR-09).

Alternatives considered: (1) Flat 500 m broadcast regardless of priority — produces alert storms in high-risk zones, accelerating B1 (alert fatigue). (2) Distance-only routing with no time or zone weights — ignores the strong temporal and spatial patterns documented in W1. (3) Manual categorisation by guards — too slow for NFR-01 (<30 s) and depends on guards being logged in. The multiplicative model was chosen because it is transparent, calibratable from real data, and can be updated periodically without redeployment (weights stored in a configuration table).

Conditions of failure: (1) Stale weights that no longer reflect current risk distribution; mitigated by quarterly recalibration. (2) GPS inaccuracy places the incident in the wrong zone; mitigated by a 20 m tolerance buffer at zone boundaries. (3) Extreme simultaneous volume (many high-P reports at once) saturates the CRITICAL queue; mitigated by RabbitMQ priority lanes and auto-scaling (see DDR-03).

How effectiveness will be measured: Average alert latency by priority tier (target: CRITICAL <15 s); false-positive rate by tier; guard response time after alert receipt.

6.3. DDR-03 — Microservices vs. Alternative Architectural Styles. — Decision: Decompose the system into five independently deployable microservices communicating over a RabbitMQ message broker.

Alternatives considered: Table 10 compares the four candidate styles.

Selection rationale: Verification and notification have non-correlated load spikes: verification peaks when many users report simultaneously; notification peaks when a major alert is dispatched. A monolith cannot scale these independently. Serverless introduces cold-start latency that could violate NFR-01 for CRITICAL alerts. SOA imposes integration overhead with no benefit over microservices at this scale.

Conditions of failure: Network partition between services could delay the `IncidentCreated` → Verification event; mitigated by RabbitMQ durability guarantees and a dead-letter queue that re-enqueues unprocessed events.

7. Operational Scenarios

The priority scoring model and verification pipeline must be grounded in concrete operational reasoning. The following three scenarios demonstrate the system's decision logic with specific inputs, calculated outcomes, and failure modes. All scenarios use the weight tables in DDR-02 (Table 9).

7.1. Scenario 1 — Evening parking-lot theft. — Context: 21:30, rainy night. A student at the parking lot witnesses a theft and submits a report with GPS coordinates placing the incident in the Parking Lot zone.

Priority calculation:

$$P = S_{\text{base}} \times W_z \times W_t = 2.0 \times 1.5 \times 1.5 = \mathbf{4.5} \rightarrow \text{CRITICAL}$$

System response: The Incident Service persists the report and publishes `IncidentCreated`. The Verification Engine routes it to the priority bypass path (CRITICAL tier skips the 45-second crowd confirmation requirement). The AI scorer runs in parallel (<3 s) and returns a plausibility score above threshold. The Dispatcher broadcasts a push alert to all users within 500 m and simultaneously sends an alert to the on-duty guard's device. *Expected latency: 8–12 s from report to first notification.*

Fallback: If rain has caused the user's GPS signal to degrade (low-accuracy reading), the 20 m zone buffer places the incident in the correct zone. If push delivery fails, the SMS fallback activates after 10 s.

7.2. Scenario 2 — SOS emergency activation. — **Context:** 20:45. A student near the dormitory area activates the SOS button (FR-08). No incident description is provided.

Priority calculation:

$$P = 4.0 \times 1.3 \times 1.5 = \mathbf{7.8} \rightarrow \text{CRITICAL (bypass)}$$

System response: The Incident Service captures GPS coordinates automatically and publishes a `SOSActivated` domain event. The Verification Engine's SOS handler bypasses all confirmation logic and routes directly to the Dispatcher within <2 s. The Dispatcher broadcasts a CRITICAL alert to the entire campus and simultaneously notifies all on-duty security guards. A 90-second ACK timer starts. If no guard acknowledges within 90 s, the integration layer pushes an escalation request to the local-authorities REST endpoint. *Expected latency: <4 s from SOS press to first guard notification.*

7.3. Scenario 3 — High-volume surge (post-event crowd). — **Context:** 22:15 following a large campus event. Thirty users simultaneously submit reports related to a physical altercation near the main entrance.

Priority calculations (representative sample):

- Assault reports at main entrance at 22:15: $P = 2.0 \times 1.0 \times 1.3 = 2.6 \rightarrow$ HIGH
- Suspicious-activity reports (noise from crowd): $P = 1.0 \times 1.0 \times 1.3 = 1.3 \rightarrow$ STANDARD

System response: RabbitMQ routes the 30 reports into two priority lanes: HIGH (assault reports) processed first, STANDARD queued behind. The spatial clustering algorithm groups geographically co-located STANDARD reports within a 30 m radius into a single composite alert, reducing 22 individual notifications to 4 composite ones. Kubernetes detects queue depth exceeding 15 events and triggers a scale-out policy on the Verification Engine, adding two pod replicas within 45 s. *Expected outcome: the 8 HIGH-priority reports are dispatched in <30 s (NFR-01 met); composite STANDARD alerts dispatched in <90 s.*

Failure mode under extreme surge: If 300+ simultaneous reports arrive (stress scenario, NFR-05), the CRITICAL/HIGH queues remain unaffected by priority isolation; STANDARD alerts may experience latency up to 5 minutes but are never lost (durable queue). Load-test targets for Phase 3 validate this envelope.

8. Risk and Complexity Management

8.1. Sensitivity parameter mitigation. — Table 11 maps each of the five sensitivity parameters identified in W1 to its quantified risk and its design-level mitigation mechanism, showing the traceability from analysis to design.

8.2. Failure mode analysis. — Table 12 maps each major architectural dependency to its failure mode, impact, and mitigation.

9. Performance Targets and Quality Assurance

9.1. Key Performance Indicators. — Table 13 defines the measurable targets. Each KPI is aligned with a specific requirement and with a W1 operational gap.

9.2. Testing strategy. — **Unit testing** validates each microservice in isolation; minimum 80% code coverage per service. **Integration testing** verifies the full event chain: Incident Service → Verification Engine → Dispatcher → Push/SMS, running against a staging environment seeded with the four observed campus zones. **Load testing** simulates a 300% concurrent-request surge (NFR-05) with target: zero timeout errors and <30 s latency at P95. A secondary test simulates the Scenario 3 volume spike (30 simultaneous reports) to validate RabbitMQ priority isolation. **Operational scenario testing** replays the three scenarios in Section 7 against the prototype, verifying that priority scores, dispatch paths, and latency targets match the specified values. **User acceptance testing (UAT)** uses a pilot cohort of students and security staff; success criterion: $\geq$85% of participants submit a report in <2 minutes and rate usability $\geq 4/5$. **Security testing** verifies TLS 1.3 compliance, Ley 1581 data controls, access-controlled audit log, and anonymous-reporter encryption.

10. Implementation Roadmap

The implementation is organised in four phases. Each phase specifies deliverables, success criteria, and the open assumptions (Section 12) that must be validated before the next phase begins.

Phase 1 — Planning and architecture (Month 1). Deliverables: finalised architecture diagrams; complete requirements traceability matrix; development environment and CI/CD pipeline setup; campus-wide mobile connectivity audit. Success criterion: every FR and NFR traceable to a W1 finding; connectivity dead-zones mapped and offline-queuing strategy confirmed. W1 assumption to close: A3 (mobile connectivity, unvalidated).

Phase 2 — Core development (Month 2). Deliverables: functional prototype with FR-01 (incident reporting) and FR-02 (geofenced alert) operational end-to-end; database schema; message broker configuration; all five microservices in development. Success

criterion: a student can submit a theft report and receive a geofenced push alert within 30 s on the staging environment.

Phase 3 — Testing and UAT (Month 3). Deliverables: unit, integration, load, and operational-scenario test reports; UAT report from pilot cohort; community engagement materials. Success criterion: NFR-01 (<30 s) and NFR-02 (99.9%) met under simulated 300% surge; ≥85% UAT success rate. W1 assumption to close: A1 (willingness to report under real conditions).

Phase 4 — Deployment and monitoring (Month 4). Deliverables: production deployment on cloud infrastructure; SIURE UD integration live; real-time monitoring dashboard active; Ley 1581 data-processing agreement signed. Success criterion: 99.9% uptime over 30-day monitoring window; guard acknowledgement rate ≥90% on CRITICAL alerts. W1 assumption to close: A2 (verification latency within 45-second budget under real load) and A5 (institutional integration).

11. Ethical and Safety Considerations

11.1. Privacy and data protection. — The CSAS processes three categories of personal data: registration information (name, institutional email, campus role); real-time location data during incident submission; and incident media (photos, video). Each category carries distinct obligations under Ley 1581 de 2012, which is directly binding on the institution.

Four compliance requirements are enforced by design: explicit informed consent at registration (purpose-specific); data used only for declared safety purposes, no third-party transfer or advertising use; user rights of access, correction, and deletion supported by the User Service; and a 12-month incident retention policy with irreversible anonymisation thereafter.

For sensitive incident types (harassment, threats), the anonymous reporting option encrypts reporter identity at rest, accessible only to system administrators under a documented, access-controlled protocol with immutable audit-log entries.

11.2. Equity and accessibility. — A mobile-first design must not create an equity gap. The SMS gateway (Twilio) functions as both a delivery fallback and a submission channel, ensuring that community members without smartphone access can both report incidents and receive alerts. The web portal provides a WiFi-connected alternative. All user-facing components use Spanish as the primary language; the onboarding flow must be completable in under two minutes (NFR-06) without presupposing advanced digital literacy.

11.3. Prevention of misuse. — Three misuse vectors are explicitly addressed: **False reports:** mitigated by the hybrid verification pipeline and human moderation (DDR-01). **Harassment through the platform:** deterred by the immutable audit log that records every moderation action. **Anonymous report flooding:** rate limiting on the anonymous path (maximum 3 anonymous reports per user per hour) prevents a single actor from overwhelming the verification queue.

11.4. Unintended consequences. — Four classes of unintended consequences, identified in the W1 revised submission, are actively mitigated: **Vigilantism:** alert content uses advisory language ("Incident reported nearby; please take precautions") and never directive language ("Approach and investigate"). **Risk profiling:** the analytics layer enforces aggregation policies; no individual movement data is exposed in heatmaps. **Reduced personal vigilance:** onboarding explicitly frames the CSAS as a complement to, not a substitute for, personal situational awareness. **Fear amplification:** the first operational period includes a communication campaign contextualising alert frequency relative to campus-wide incident prevalence.

12. Open Assumptions Requiring Validation

Table 14 restates the five assumptions from the W1 revised submission, updated with the specific validation action assigned to each implementation phase.

13. Conclusions

This document has translated the empirical findings of Workshop No. 1 into a complete, traceable systems design specification for the CSAS. Five conclusions summarise the design-phase contributions.

The design is system-driven, not feature-driven. Every functional requirement is traceable to a W1 empirical finding (Table 2); every architectural choice is justified by a Design Decision Record (Section 6) that documents alternatives considered and conditions of failure.

The priority dispatch model is now formally specified. The multiplicative model $P = S_{\text{base}} \times W_z \times W_t$ converts the W1 observation-based risk findings into an operational mechanism with explicit weights, thresholds, and calibration policy. The model is transparent, updatable, and measurable.

The verification pipeline resolves the B1–R1 tension. The hybrid AI + human design (DDR-01) is the only configuration that simultaneously achieves low latency (NFR-01), high accuracy, and resilience to moderator unavailability. This conclusion is supported by the alternative evaluation in Table 8.

Three concrete scenarios confirm the design's operational validity. The scenarios in Section 7 demonstrate that the system produces correct, calibrated responses to the three most important risk events identified in W1: an evening parking-lot incident, a life-safety SOS, and a high-volume post-event surge.

The largest remaining risks are behavioural, not technical. The adoption gap (+48 pp to reach the 60% target) cannot be closed by technical means alone; it requires institutional endorsement, community engagement, and a sustained communication strategy. This is the primary operational risk for the next implementation phase.

References

[1] C. A. Sierra, *TeamWork as Computer Engineers — CS Collaboration Guidelines*. Bogotá: Universidad Distrital Francisco José de Caldas, 2026.

[2] P. Checkland, *Systems Thinking, Systems Practice*. New York: John Wiley & Sons, 1981.

[3] D. H. Meadows, *Thinking in Systems: A Primer*. White River Junction, VT: Chelsea Green Publishing, 2008.

[4] ISO/IEC/IEEE, *Systems and Software Engineering — System Life Cycle Processes*, Std. 15288:2015, 2015.

[5] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. Thousand Oaks, CA: SAGE Publications, 2018.

[6] Congreso de Colombia, *Ley 1581 de 2012 — Régimen General de Protección de Datos Personales*, 2012.

[7] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Boston: Addison-Wesley, 2013.

[8] N. Dragoni et al., "Microservices: Yesterday, Today, and Tomorrow," *Present and Ulterior Software Engineering*, pp. 195–216, 2017.

[9] J. D. Sterman, *Business Dynamics: Systems Thinking and Modeling for a Complex World*. Boston: McGraw-Hill, 2000.

Table 6: Microservices — Responsibilities and Domain Events

Service	Responsibility	Domain events
Incident	Receives, validates, and persists reports; enriches with geospatial metadata.	Publishes `IncidentCreated`
Verification	Orchestrates the hybrid AI + human pipeline (DDR-01). Adaptive threshold, priority bypass for SOS and high-severity types.	Consumes `IncidentCreated`; publishes `IncidentVerified`
Dispatcher	Computes priority score (DDR-02); routes to push / SMS.	Consumes `IncidentVerified`; publishes `AlertDispatched`
User	Manages registration, roles, alert preferences; implements alert-fatigue dampening.	Consumes `AlertDispatched`
Analytics	Aggregates domain events into the data warehouse; produces heatmaps.	Consumes all events

Table 7: Recommended Technology Stack

Component	Technology	Justification
Mobile app	React Native	Single codebase Android/iOS; ≤3-interaction reporting (NFR-06).
API gateway	Node.js / Nginx	High I/O throughput; handles concurrent submissions and rate limiting.
Backend services	Node.js	Asynchronous, event-driven I/O supports <30 s latency (NFR-01).
Verification Engine	Python / FastAPI	Python ML ecosystem for AI plausibility scorer (DDR-01).
Message broker	RabbitMQ	Priority-queue support; decouples ingestion from verification (mitigates volume spike risk).
Database	PostgreSQL + PostGIS	Native geospatial queries for zone-weighted routing and heatmaps.
Push notifications	Firebase FCM	Cross-platform; supports 99.9% delivery (NFR-02).
SMS fallback	Twilio API	Guaranteed delivery for users without push capability; equity of access.
Infrastructure	AWS / Azure + Kubernetes	Container orchestration enables auto-scaling (NFR-03, NFR-05).

Table 8: DDR-01 — Evaluation of Verification Alternatives

Option	**Latency**	**Accuracy**	**Failure resilience**	**Alert-fatigue risk**	**Cost**
AI-only	Very low	Medium (degrades silently)	Low (no fallback)	High if model errors	Low
Human-only	High (>2 min estimated)	High	High	Low	High (staffing)
Crowd-only (≥3 confirmations)	Variable	Medium	Low (sparse off-hours)	Medium	Very low
Hybrid AI + human (chosen)	Low (AI prefilters)	High (human corrects AI)	High (human override)	Low (AI suppresses noise)	Medium

Table 9: DDR-02 — Priority Score Weight Tables

Severity base score S_{base}		
SOS / life-safety		4.0
Theft / assault		2.0
Harassment		1.5
Suspicious activity		1.0
Zone weight W_z (W1 observation data)		
Parking lot (night)	Highest-risk across all sessions	1.5
Dormitory area	Poor lighting; latent risk	1.3
Main entrance	High foot traffic; deterrent effect	1.0
Library area	High activity; no suspicious events	1.0
Time weight W_t (W1 observation + survey)		
18:00–22:00	All suspicious events in this window	1.5
22:00–06:00	Very low foot traffic; high risk	1.3
06:00–18:00	Normal activity; lower risk	1.0

Priority tier and dispatch action		
$P \geq 4.0$	CRITICAL	Immediate dispatch; 500 m alert; auth. escalation path
$2.5 \leq P < 4.0$	HIGH	Standard dispatch; 500 m alert; on-duty guard
$1.0 \leq P < 2.5$	STANDARD	Queued dispatch; 250 m alert
$P < 1.0$	SUPPRESSED	Logged; not broadcast

Table 10: DDR-03 — Evaluation of Architectural Style Alternatives

Style	**Independent scaling**	**Deployment complexity**	**Failure isolation**	**Team fit**
Monolith	None (all-or-nothing)	Low	Low (one failure cascades)	Low (parallel dev difficult)
Serverless (FaaS)	High	Medium	High	Medium (cold-start latency risks NFR-01)
SOA (heavyweight)	Medium	High	Medium	Low (requires ESB)
Microservices (chosen)	Per-service	Medium	High	High (teams own services)

Table 11: Sensitivity Parameter Risk and Design Mitigation

Parameter	**Risk**	**Design mechanism**
Report volume spike	Verification engine saturates; >30 s latency.	RabbitMQ priority lanes; auto-scaling on Verification pods; SOS and CRITICAL bypass standard queue.
Low user participation	Real incidents undetected; R1 loop stalls.	≤3-interaction interface (FR-01); anonymous reporting option; community engagement module; SIURE integration as institutional endorsement.
Night-time concentration	Higher alert volume; fewer moderators online.	Time weights elevate night-hour priority; on-call escalation path; 45-second moderator timeout with auto-escalation.
High-risk zone clustering	Alert storms in parking lot and dormitory.	Zone-based rate limiting; spatial clustering aggregates co-located simultaneous reports (max 1 composite per 30 m / 60 s).
Adverse weather	Visibility drops; GPS degrades; incident risk rises.	Weather API feeds risk scoring model; adverse conditions elevate W_t by one tier in affected zones; GPS 20 m boundary tolerance.

Table 12: Failure Mode Analysis

Failure mode	**Impact**	**Mitigation**	**Recovery time target**
API gateway outage	All submissions fail	Multi-region deployment + health checks; mobile app queues reports locally.	<60 s automatic failover
Verification Engine overload	Alert latency exceeds NFR-01	Auto-scaling pods; CRITICAL/SOS bypass path is unaffected.	<45 s pod spin-up
Push delivery failure	Users miss critical alerts	SMS fallback activates after 10 s push timeout.	<10 s
Database unavailability	Incident data loss risk	Primary-replica replication; write-ahead logging; daily backups.	Zero data loss; <120 s reconnect
AI scorer false positive	Legitimate incident blocked	Human review queue; 90-second escalation timer if no moderator.	<90 s override
Mobile dead zone	Submission fails	Offline queue in app; SMS submission channel as fallback.	On reconnect

Table 13: Key Performance Indicators

KPI	Target	Req.	W1 gap	Freq.
Alert latency (end-to-end)	<30 s P95	NFR-01	Response time −6.4 min	Daily
Delivery success rate	≥99.9%	NFR-02	Delivery −12.9 pp	Daily
System availability	≥99.9%	NFR-03	—	Monthly
Incident record rate	≥85% logged	FR-07	Completeness +44 pp	Weekly
User adoption	≥60% students	—	Adoption +48 pp	Monthly
Verification latency	<45 s	DDR-01	—	Daily
False-positive rate	<5% verified alerts	DDR-01	B1 loop mitigation	Daily
Scalability (surge)	300% surge, 0 timeouts	NFR-05	—	Monthly load test

Table 14: Open Assumptions and Validation Plan

#	Assumption	Current evidence	Status	Validation action
A1	Users will report if friction is reduced	100% of incident-experienced survey respondents willing to adopt (W1, $n = 20$)	Plausible	Phase 3 UAT: measure actual submission rate in pilot
A2	Verification can operate within 45 s budget	Scenario analysis; no empirical measurement	Untested	Phase 4 load test under real conditions
A3	Mobile connectivity reliable across all zones	Not assessed in W1	Unvalidated	Phase 1: connectivity audit before deployment
A4	False-positive rate below B1 threshold	Theoretical, DDR-01 design	Untested	Phase 3: measure FP